Software Engineer: Case Study Assignment

Haqdarshak · Scheme Knowledge Base
Thank you for your interest in joining Haqdarshak. This assignment is built around a real problem we are working on right now. It gives you a chance to show how you think, and gives us a feel for what working together would be like.

About Haqdarshak

Haqdarshak, founded in 2015, helps citizens at the last mile find and apply for government welfare schemes. Trained community agents, called Haqdarshaks and mostly women, use a mobile app to check which schemes a family is eligible for and then help them apply. Through this network, Haqdarshak has helped millions of citizens across India access their entitlements.

The problem we are solving

All of this depends on one thing being right: our database of about 5,000 central and state schemes. Each scheme has its benefits, eligibility rules, required documents and application steps.
Today, researchers keep this database up to date by reading government PDFs, department websites and the national MyScheme catalogue, and typing the details in by hand. The work is tracked across a portal, an Excel sheet and WhatsApp. It is slow, it is hard to tell where a piece of information came from, and changes get missed. When that happens, a wrong amount or an expired deadline reaches an agent, and then a citizen.
We are building a **Scheme Knowledge Base** to fix this. AI prepares a first draft of each scheme record, a person reviews it, and only approved versions reach the agent app, the citizen chatbot and our partners.

Two principles we hold to

- **A person always checks before anything goes live.** Whatever the AI produces is a draft. Someone reviews and approves it before agents or citizens see it.
- **Every fact points back to its source.** Each piece of information should show the document or page it came from. If you cannot show a source, leave the field empty and say why, rather than filling it in.

How this assignment works

- **Pick one** of the three problems in your brief. You do not need to attempt all three.
- **You will not receive a separate dataset or reference pack.** Use the context in this brief, make reasonable assumptions and create synthetic examples or sample data wherever useful. Please explain your assumptions and why you made them.
- **Small examples are enough.** A few well-chosen examples that show the tricky cases tell us more than a large dataset.
- **Use any language, framework or tool you like, including coding agents and AI assistants** such as Claude Code, Cursor, Copilot or ChatGPT. You are welcome to use them to build and demonstrate your solution. Tell us where and how you used them, and be ready to explain or change any part of your work yourself.
- **Say what you do not know.** Where information is missing, state your assumption or flag the gap.

We would much rather see a clear assumption than a confident guess.

- **We do not expect a production-ready solution.** Tell us what you chose to focus on and what you left out on purpose.

What to submit

Role	What to send	Suggested effort	Walkthrough
Software Engineer	A code repository (link or zip) with a README covering your assumptions, how to run it and what you left out	4–6 hours	10 minutes, live or recorded

Deadline: Please submit within 3 days of receiving the assignment. If you are currently working or need more time, you can request an extension of up to 2 additional days.
Where to send it: Submit your work to [hiring contact/email] with the subject line: Software Engineer Case Study – [Your Name].

In your walkthrough, please cover

1. One decision you made, and the option you decided against.
2. Where your work falls short, and how someone would notice.
3. The main assumptions you made, and why.
4. The "Also explain" point and the question about how you work, for the problem you chose.

~~How we will look at your work¶~~

~~We look at four things. We are not scoring polish or volume.¶~~

- ~~**How you approach the problem:** how you break it down and deal with messy, real-world detail.¶~~
- ~~**The decisions you make:** what you chose, why, and what you gave up.¶~~
- ~~**How you explain:** whether someone outside your field could follow you, and how honest you are about limits.¶~~
- ~~**How you work with others:** how you handle feedback, disagreement and the knowledge of people in the field.~~

How we will use your submission

We will use your submission only to evaluate your application. If we would like to reuse any part of it, we will discuss this with you first and agree on a fixed fee.

Software Engineer: case study options

Choose **one** of S1, S2 or S3.

Field format

To keep solutions comparable, please represent each scheme field as a small record with the parts below. JSON, a database table or anything similar is fine.

Part	What it holds
scheme_ref	Which scheme this belongs to (an ID, or the source

	document if there is no ID yet)
field	The field name, for example benefit_amount or deadline
value	The value, or empty if not found
evidence	The exact text it came from, where in the source (page or section), the source link and the date it was captured. For a synthetic source, use a source name or placeholder reference instead of a link
confidence	How sure you are: a number from 0 to 1, or high / medium / low
status	found, not found, or conflict. A conflict means sources disagree: keep every value and its source
note	Optional. A short reason, for example why something could not be mapped

If your solution uses an AI model, note which model and version, and roughly what it would cost per scheme.

S1 · Review and publish backend

The problem

Scheme records change all the time: amounts get revised and deadlines move. Agents in the field must keep seeing the approved version while a correction is being prepared and checked. Build the backend that makes this safe.

What to build

1. Support two roles. State reviewers create and edit drafts; Central reviewers approve and publish. The person who creates a new scheme draft cannot also approve it.
2. A record moves from draft to in review to published, or is sent back with a comment.
3. Publishing locks that version. Editing a live scheme starts a new draft, and the read API keeps serving the last published version until the new one is approved.
4. Record every field change: who made it, when, the old value, the new value and the source.
5. Eligibility rules can be shared across schemes, for example "annual family income below ₹2.5 lakh". Changing a shared rule flags every scheme that uses it for review. It never changes a published scheme silently.
6. Demonstrate this with a handful of synthetic schemes and tests for each rule above. Any stack is fine; we use PostgreSQL, so that is preferred if you are comfortable with it.

Also explain in your walkthrough

How you would explain versioning to a non-technical programme head, and what you expect to break first at 5,000 schemes.

A question about how you work

Two days before go-live, you find a bug in a teammate's code. A proper fix will take a day. What do you do, and who do you tell?

S2 · Field inputs to schema

The problem

Extracted text is messy. A benefit might read "₹6,000 a year in three instalments", and eligibility might read "SC/ST families with income under ₹2.5 lakh". To check eligibility automatically, this has to become

structured data.

What to build

1. Create a set of messy field values that reflects what real scheme documents look like, along with a small list of eligibility concepts (such as age, income, social category and state) and benefit categories.
2. Turn benefits into rows: category, type, amount, currency, frequency and number of instalments.
3. Turn eligibility into conditions: concept, operator and value, combined with AND / OR.
4. Send anything you cannot map confidently to a review list, with the reason. Do not guess.
5. Write tests for the tricky cases.

Also explain in your walkthrough

Where you used plain rules, where you would use a model, and why.

A question about how you work

You notice the concept list is missing something obvious. Do you add it yourself? What do you do?

S3 · Source change watcher

The problem

Government departments update scheme pages without notice. We need to know when a page changes in a way that matters, without flooding reviewers with alerts every time a banner or a date stamp changes.

What to build

1. Create synthetic page snapshots, or use public pages, that show both cosmetic and meaningful changes.
2. Store a dated snapshot of each page and re-check pages on a schedule.
3. Ignore cosmetic changes. When something meaningful changes, such as an amount, deadline or eligibility rule, create a review item with a readable before-and-after.
4. Handle dead links and pages that move.

Also explain in your walkthrough

How you kept false alarms low, and what you log.

A question about how you work

A state department's website blocks automated access. What do you do?